

PERTEMUAN 11

ANIMASI DASAR DENGAN TEKNIK INTERPOLASI

Kemampuan Akhir yang Diharapkan (KAD):

Mahasiswa mampu mengembangkan animasi komputer dasar menggunakan teknik interpolasi.

Pokok Bahasan:

1. Konsep dasar animasi dalam grafika komputer.
2. Prinsip interpolasi dalam pergerakan animasi.
3. Menerapkan teknik interpolasi linier (linear interpolation) dan non-linier untuk menghasilkan animasi dasar.
4. Mengimplementasikan animasi menggunakan Java Processing.
5. Soal Latihan

11.1 Konsep Dasar Animasi

Animasi dalam grafika komputer adalah proses menciptakan ilusi gerakan dengan menampilkan rangkaian gambar (frame) secara berurutan dalam waktu tertentu. Jika gambar-gambar tersebut ditampilkan cukup cepat, biasanya lebih dari 24 frame per detik (fps) maka mata manusia akan menangkapnya sebagai satu gerakan yang halus, bukan rangkaian gambar terpisah. Dalam grafika komputer, animasi tidak dibuat secara manual frame demi frame seperti dalam animasi tradisional, melainkan dihasilkan secara matematis melalui transformasi (translasi, rotasi, skala) dan interpolasi di antara titik-titik kunci (keyframe).

Teknik animasi yang baik membutuhkan transisi halus antar frame, yang dapat dicapai dengan interpolasi. Interpolasi digunakan untuk menghitung nilai di antara dua posisi (atau keadaan) objek dalam rentang waktu tertentu. Dengan demikian, pergerakan atau transformasi terlihat halus dan realistis.

Elemen Utama dalam Animasi Komputer:

Elemen	Penjelasan
Objek (Model)	Entitas visual yang dianimasikan. Bisa berupa titik, garis, bentuk 2D, atau model 3D.
Frame	Satu unit gambar tunggal dalam urutan animasi.
Keyframe	Frame penting yang menentukan posisi atau keadaan utama objek pada waktu tertentu. Frame di antaranya dihitung dengan interpolasi.
Timeline (Sumbu waktu)	Representasi waktu animasi; menunjukkan kapan perubahan posisi atau transformasi terjadi.
Interpolator	Fungsi atau algoritma yang menghitung nilai di antara dua keyframe (misal: linear, quadratic, cubic).

Prinsip Waktu dalam Animasi:

Waktu adalah elemen terpenting dalam animasi komputer. Tanpa pengaturan waktu, objek hanya berpindah posisi tanpa makna visual.

- **Frame rate (fps)** → jumlah frame yang ditampilkan per detik.

Umumnya digunakan:

- 24 fps → film sinematik
- 30 fps → animasi digital / game ringan

- 60 fps → game modern, simulasi real-time
- **Durasi animasi** dihitung dari:

$$\text{durasi} = \frac{\text{jumlah frame}}{\text{frame rate}}$$

- **Time-based animation (delta time):**

Agar animasi berjalan sama di komputer cepat maupun lambat, perubahan posisi dihitung berdasarkan waktu aktual, bukan frame.

$$\text{posisi}(t) = \text{posisi}_{awal} + \text{kecepatan} \times \Delta t$$

Prinsip Gerakan (Animation Principles)

Dalam dunia animasi (baik manual maupun komputer), terdapat **12 prinsip dasar animasi** (dirumuskan oleh animator Disney, Frank Thomas & Ollie Johnston) yang juga berlaku dalam animasi komputer.

Berikut prinsip-prinsip yang paling relevan untuk grafika komputer:

Prinsip	Penjelasan	Implementasi Komputer
Timing	Mengatur kecepatan gerakan; seberapa cepat atau lambat objek bergerak.	Diatur dengan parameter waktu atau fungsi easing.
Ease In / Ease Out	Gerakan lambat di awal, cepat di tengah, lalu melambat di akhir.	Gunakan interpolasi non-linier (misal fungsi kuadratik).
Anticipation	Gerakan persiapan sebelum aksi utama.	Tambahkan gerakan kecil sebelum animasi utama.
Follow Through	Bagian tubuh atau objek lain tetap bergerak setelah aksi utama berhenti.	Simulasi menggunakan fisika atau delay antar bagian.
Squash and Stretch	Perubahan bentuk untuk menonjolkan bobot dan elastisitas.	Modifikasi skala objek selama animasi.
Staging	Menentukan penempatan objek agar fokus perhatian jelas.	Atur posisi kamera dan pencahayaan.

Teknik Animasi Komputer

Berikut adalah teknik-teknik utama dalam pembuatan animasi digital:

1. Keyframe Animation

Animator menentukan pose-pose utama (keyframe), komputer menghitung frame di antaranya dengan interpolasi.

Paling umum digunakan pada software seperti Blender, Maya, atau After Effects.

2. Procedural Animation

Gerakan dihasilkan oleh algoritma matematika atau aturan fisika.

Contoh: simulasi ombak, partikel hujan, gerak acak.

3. Skeletal Animation

Menggunakan struktur tulang (bone) untuk menggerakkan model karakter.

Gerakan dihasilkan dengan transformasi tulang dan interpolasi antar pose.

4. Morph Target Animation (Shape Interpolation)

Bentuk model diubah dari satu mesh ke mesh lain secara bertahap (misal ekspresi wajah).

5. Physics-based Animation

Menggunakan hukum fisika (gaya, gravitasi, momentum).

Biasanya disimulasikan dengan integrator (Euler, Verlet, dll).

11.2 Konsep Dasar Interpolasi dalam Animasi

Interpolasi adalah metode untuk menentukan nilai di antara dua titik data. Dalam konteks animasi, interpolasi digunakan untuk menghitung posisi, rotasi, skala, atau warna objek pada waktu tertentu di antara dua keyframe. Ketika seorang animator menentukan posisi objek pada **keyframe awal** dan **keyframe akhir**, komputer menggunakan **fungsi interpolasi** untuk menghitung posisi objek pada frame-frame di antaranya.

Interpolasi adalah inti dari animasi digital. Tujuannya adalah **mengisi nilai di antara dua keyframe** secara matematis agar transisi halus.

Tujuan interpolasi:

- Menghasilkan **transisi yang mulus** antara dua keadaan.
- **Menghemat waktu** animasi (tidak perlu menggambar setiap frame).
- Menciptakan **ilusi gerakan alami** menggunakan fungsi matematika.

Interpolasi bekerja berdasarkan nilai parameter waktu t dengan rentang:

$$0 \leq t \leq 1$$

- $t=0 \rightarrow$ keadaan awal (keyframe 1)
- $t=1 \rightarrow$ keadaan akhir (keyframe 2)
- Nilai antara 0 dan 1 menentukan posisi atau nilai di antaranya

Secara umum, interpolasi dapat dituliskan sebagai:

$$f(t) = (1 - t) \cdot P_0 + t \cdot P_1$$

di mana:

P_0 = posisi awal

P_1 = posisi akhir

t = waktu normalisasi (normalized time)

$t \in [0,1]$ = parameter waktu (0 awal, 1 akhir)

Misalnya posisi objek berubah dari (100, 200) $\rightarrow$ (400, 200) dalam 2 detik.

Dengan interpolasi linier:

$$x(t) = x_0 + (x_1 - x_0) \cdot t$$

Hasil:

- $t = 0 \rightarrow$ posisi 100
- $t = 0.5 \rightarrow$ posisi 250
- $t = 1 \rightarrow$ posisi 400

Fungsi interpolasi lain (easing) bisa digunakan untuk menghasilkan efek alami:

- **ease-in:** gerak mulai lambat (t^2)
- **ease-out:** gerak melambat di akhir ($\sqrt{t}$)
- **ease-in-out:** kombinasi keduanya ($3t^2 - 2t^3$)

Interpolasi dapat diterapkan pada berbagai atribut animasi, bukan hanya posisi:

Atribut	Contoh Interpolasi
Posisi (x, y, z)	Gerakan translasi objek
Rotasi	Perputaran objek di sekitar sumbu
Skala	Membesarkan atau mengecilkan objek
Warna	Transisi dari satu warna ke warna lain
Transparansi (Alpha)	Efek fade-in / fade-out

11.2.1 Linear Interpolation (Lerp)

Interpolasi linear (sering disingkat *Lerp* dari *Linear Interpolation*) adalah metode paling sederhana untuk memperkirakan nilai di antara dua titik data (keyframe) dengan **kecepatan perubahan yang konstan**.

Dalam konteks animasi komputer, interpolasi linear digunakan untuk menghitung posisi, warna, rotasi, atau parameter lainnya dari suatu objek antara dua keadaan utama selama selang waktu tertentu.

Rumus:

$$P(t) = (1 - t) \cdot P_0 + t \cdot P_1$$

di mana:

P_0 = posisi awal

P_1 = posisi akhir

$t \in [0,1]$ = parameter waktu (0 awal, 1 akhir)

Jika $t=0.5$, maka nilai berada tepat di tengah antara P_0 dan P_1 .

Contoh:

Jika $P_0=0$, $P_1=100$, dan $t=0.25$

Maka $P(0.25)=(1 - 0.25) \times 0 + 0.25 \times 100 = 25$

Artinya, pada waktu 25% dari total durasi, objek sudah mencapai posisi 25 dari 100.

Visualisasi: Garis lurus dari titik awal ke akhir, gerakannya rata, tidak ada percepatan atau perlambatan.

Untuk mendapatkan animasi yang berjalan halus, nilai t biasanya dihitung berdasarkan waktu nyata (real-time):

$$t = \frac{\text{waktu sekarang} - \text{waktu mulai}}{\text{durasi total}}$$

Sehingga animasi akan tetap **konstan** di semua perangkat, tidak bergantung pada frame rate.

Pseudocode umum untuk interpolasi linear:

```
function Lerp(a, b, t):
    return (1 - t) * a + t * b
```

Algoritma ini digunakan di banyak sistem animasi, termasuk game engine seperti Unity, Unreal Engine, Blender, dan library grafis seperti Processing atau WebGL.

Contoh 1: Animasi Posisi Sederhana

Objek (lingkaran) bergerak dari kiri ke kanan dengan kecepatan konstan.

```
float x0 = 100;    // posisi awal
float x1 = 500;    // posisi akhir
float y = 250;     // posisi vertikal tetap
```

```
float t = 0; // parameter waktu
float speed = 0.01; // kecepatan interpolasi

void setup() {
  size(600, 400);
  noStroke();
  fill(255, 100, 100);
}

void draw() {
  background(0);
  // Hitung posisi menggunakan interpolasi linear
  float x = lerp(x0, x1, t);
  // Gambar objek
  ellipse(x, y, 40, 40);
  // Perbarui nilai t
  t += speed;
  // reset agar animasi kembali ke awal
  if (t > 1) t = 0;
}
```

Penjelasan:

- Fungsi `lerp(a, b, t)` di Processing mengimplementasikan rumus interpolasi linear.
- Nilai `t` naik dari 0 ke 1 → posisi berubah dari `x0` ke `x1`.
- Setelah mencapai 1, posisi objek kembali ke posisi awal.

Contoh 2: Animasi Posisi Sederhana Bolak-Balik

Objek (lingkaran) bergerak dari kiri ke kanan dengan kecepatan konstan.

```
float x0 = 100; // posisi awal
float x1 = 500; // posisi akhir
float y = 250; // posisi vertikal tetap
float t = 0; // parameter waktu
float speed = 0.01; // kecepatan interpolasi

void setup() {
  size(600, 400);
  noStroke();
  fill(255, 100, 100);
}

void draw() {
  background(30);

  // Hitung posisi menggunakan interpolasi linear
  float x = lerp(x0, x1, t);

  // Gambar objek
  ellipse(x, y, 40, 40);

  // Perbarui nilai t
  t += speed;
  if (t > 1) {
    // reset agar animasi bolak-balik
    float temp = x0;
    x0 = x1;
    x1 = temp;
    t = 0;
  }
}
```

Penjelasan:

- Fungsi `lerp(a, b, t)` di Processing mengimplementasikan rumus interpolasi linear.
- Nilai `t` naik dari 0 ke 1 → posisi berubah dari `x0` ke `x1`.
- Setelah mencapai 1, posisi ditukar agar objek bergerak bolak-balik.

Contoh 3: Interpolasi Linear Warna

Selain posisi, interpolasi linear juga bisa diterapkan untuk transisi warna antar dua warna.

```
color c1 = color(255, 0, 0); // merah
color c2 = color(0, 0, 255); // biru
float t = 0;
float speed = 0.01;

void setup() {
  size(400, 400);
  noStroke();
}

void draw() {
  // Gunakan lerpColor() untuk interpolasi antar warna
  background(lerpColor(c1, c2, t));

  t += speed;
  if (t > 1) {
    // Tukar warna agar berulang
    color temp = c1;
    c1 = c2;
    c2 = temp;
    t = 0;
  }
}
```

Penjelasan:

- Fungsi `lerpColor()` menghitung interpolasi RGB antar dua warna.
- Gerakan nilai `t` menciptakan efek transisi halus dari merah ke biru.

Contoh 4: Interpolasi Linear untuk Rotasi dan Skala

```
float angle0 = 0;
float angle1 = PI * 2;
float scale0 = 50;
float scale1 = 150;
float t = 0;
float speed = 0.01;

void setup() {
  size(600, 400);
  rectMode(CENTER);
}

void draw() {
  background(0);
  translate(width/2, height/2);

  // Interpolasi rotasi dan skala
  float angle = lerp(angle0, angle1, t);
  float s = lerp(scale0, scale1, t);

  rotate(angle);
```

```
fill(255, 200, 50);
rect(0, 0, s, s);

t += speed;
if (t > 1) t = 0;
}
```

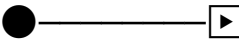
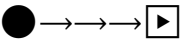
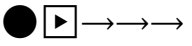
Hasil: Kotak akan berputar dan membesar secara konstan menggunakan interpolasi linier.

11.2.2 Quadratic Interpolation

Non-linear interpolation digunakan ketika kita ingin memberikan **efek percepatan (acceleration)** atau **perlambatan (deceleration)** pada gerakan. Menggunakan fungsi kuadratik untuk membuat gerakan lebih halus (misalnya easing in/out). Menggunakan fungsi kuadratik seperti t^2 , $\sqrt{t}$, atau kombinasi untuk mengubah kecepatan.

Jenis	Fungsi	Karakteristik
Ease-In	$f(t) = t^2$	Gerakan mulai lambat lalu makin cepat
Ease-Out	$f(t) = 1 - (1 - t)^2$ atau $f(t) = t \times (2 - t)$	Gerakan mulai cepat lalu melambat
Ease-In-Out	$f(t) = 2t^2$ (untuk $t < 0.5$) $f(t) = -1 + (4 - 2t)t$ (untuk $t \geq 0.5$)	Kombinasi lambat-cepat-lambat

Contoh visual perbandingan:

Linear	Ease-In	Ease-Out	Ease-In-Out
			

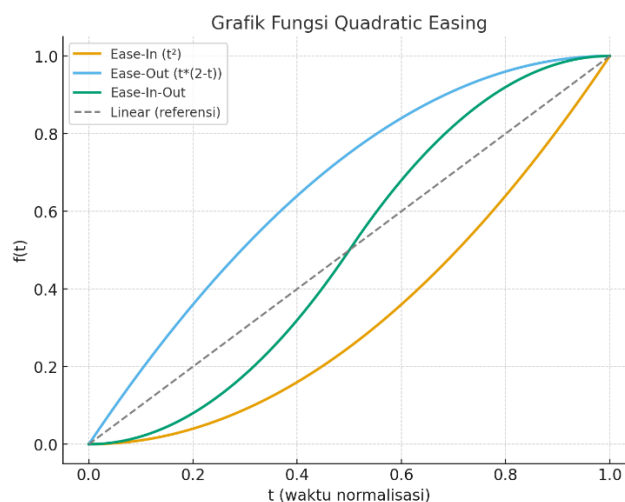
Untuk mendapatkan nilai interpolasi dengan efek quadratic, kita modifikasi fungsi dasar Lerp:

$$P(t) = (1 - f(t)) \cdot P_0 + f(t) \cdot P_1$$

di mana $f(t)$ adalah fungsi easing non-linear.

Misalnya:

- Untuk ease-in: $f(t) = t^2$
- Untuk ease-out: $f(t) = t(2 - t)$



Berikut ilustrasi grafik dari tiga fungsi **Quadratic Easing**:

- **Ease-In (t^2)** → kurva naik perlahan di awal, lalu makin cepat.
- **Ease-Out ($t(2-t)$)** → cepat di awal, lalu melambat di akhir.
- **Ease-In-Out** → lambat di awal dan akhir, cepat di tengah.

Garis abu-abu menunjukkan **interpolasi linear (referensi)** yang bergerak dengan kecepatan konstan.

Contoh 5: Ease-In Quadratic

```
float x0 = 100;    // posisi awal
float x1 = 500;    // posisi akhir
float y = 250;     // posisi vertikal tetap
float t = 0;       // parameter waktu
float speed = 0.01; // kecepatan interpolasi

void setup() {
  size(600, 400);
  noStroke();
  fill(255, 100, 100);
}

void draw() {
  background(30);

  // Fungsi quadratic ease-in: f(t) = t^2
  float f = t * t;
  float x = lerp(x0, x1, f);
  // Gambar objek
  ellipse(x, y, 40, 40);
  // Perbarui nilai t
  t += speed;
  // reset agar animasi kembali ke awal
  if (t > 1) t = 0;
}
```

Penjelasan:

- Nilai kecil t menghasilkan $f(t)$ yang lebih kecil (karena dikuadratkan).
- Akibatnya, pergerakan di awal lebih lambat, lalu semakin cepat mendekati akhir.

Contoh 6: Ease-Out Quadratic

```
float x0 = 100;    // posisi awal
float x1 = 500;    // posisi akhir
float y = 250;     // posisi vertikal tetap
float t = 0;       // parameter waktu
float speed = 0.01; // kecepatan interpolasi

void setup() {
  size(600, 400);
  noStroke();
  fill(255, 100, 100);
}

void draw() {
  background(30);

  // Fungsi quadratic ease-out: f(t) = t * (2 - t)
  float f = t * (2 - t);
  float x = lerp(x0, x1, f);
  // Gambar objek
}
```



```

ellipse(x, y, 40, 40);
// Perbarui nilai t
t += speed;
// reset agar animasi kembali ke awal
if (t > 1) t = 0;
}

```

Penjelasan:

- Di awal, t masih kecil → gerak cepat.
- Mendekati akhir, perubahan semakin kecil → gerak melambat.

Contoh 7: Ease-In-Out Quadratic

```

float x0 = 100;    // posisi awal
float x1 = 500;    // posisi akhir
float y = 250;     // posisi vertikal tetap
float t = 0;       // parameter waktu
float speed = 0.01; // kecepatan interpolasi

void setup() {
  size(600, 400);
  noStroke();
  fill(255, 100, 100);
}

void draw() {
  background(30);

  float f;
  if (t < 0.5) {
    f = 2 * t * t; // ease-in
  } else {
    f = -1 + (4 - 2 * t) * t; // ease-out
  }

  float x = lerp(x0, x1, f);
  // Gambar objek
  ellipse(x, y, 40, 40);
  // Perbarui nilai t
  t += speed;
  // reset agar animasi kembali ke awal
  if (t > 1) t = 0;
}

```

Penjelasan:

- Di awal ($t < 0.5$), gerak lambat → percepatan.
- Di akhir ($t > 0.5$), gerak melambat → perlambatan.
- Hasilnya adalah **gerakan halus dan alami** seperti objek yang berhenti dengan lembut.

Contoh 8: Bola Memantul dengan Interpolasi Quadratic

Simulasi bola yang melambat di puncak dan cepat saat jatuh bisa menggunakan fungsi quadratic:

$$y(t) = h_{max} \times (1 - (2t - 1)^2)$$

```

float t = 0;
float speed = 0.02;

```

```
float y;

void setup() {
  size(400, 400);
  noStroke();
  fill(255, 100, 100);
}

void draw() {
  background(30);

  // quadratic bounce
  float f = 1 - (2*t - 1)*(2*t - 1);
  y = map(f, 0, 1, height - 50, 50);

  ellipse(width/2, y, 40, 40);

  t += speed;
  if (t > 1) t = 0;
}
```

11.2.3 Cubic Interpolation (Bezier Curve)

Pada pembahasan sebelumnya, kita telah mempelajari **interpolasi quadratic**, yang memberikan efek percepatan dan perlambatan sederhana. Namun, untuk menciptakan **gerakan yang lebih halus dan ekspresif**, diperlukan fungsi non-linear dengan kurva lebih kompleks, seperti **fungsi kubik (cubic)** dan **fungsi berbasis gaya fisika (bounce, elastic)**. Interpolasi kubik digunakan dalam hampir semua **animasi profesional**, baik di game engine, film animasi, maupun antarmuka pengguna (UI/UX).

Interpolasi kubik menggunakan fungsi pangkat tiga (t^3) untuk menggambarkan perubahan waktu. Bentuk umumnya:

$$f(t) = t^3$$

Namun dalam konteks easing, fungsi ini dimodifikasi untuk menghasilkan tiga varian utama:

Jenis	Rumus Matematis	Karakteristik Gerakan
Ease-In Cubic	$f(t) = t^3$	Lambat di awal, cepat di akhir
Ease-Out Cubic	$f(t) = 1 - (1 - t)^3$ atau $f(t) = (t - 1)^3 + 1$	Cepat di awal, lembut di akhir
Ease-In-Out Cubic	$f(t) = 4t^3$ ($t < 0.5$) $f(t) = 1 - (-2t + 2)^3 / 2$ ($t \geq 0.5$)	Lembut di awal dan akhir, cepat di tengah

Kelebihan:

- Memberikan kontrol tinggi terhadap kehalusan gerakan.
- Digunakan dalam software animasi profesional dan CSS animation.

Contoh 9: Ease-In Cubic

```
float x0 = 100;    // posisi awal
float x1 = 500;    // posisi akhir
float y = 250;     // posisi vertikal tetap
float t = 0;       // parameter waktu
float speed = 0.01; // kecepatan interpolasi
```

```

void setup() {
  size(600, 400);
  noStroke();
  fill(255, 100, 100);
}

void draw() {
  background(30);

  // Fungsi cubic ease-in
  float f = pow(t, 3);
  float x = lerp(x0, x1, f);

  ellipse(x, y, 40, 40);
  t += speed;
  if (t > 1) t = 0;
}

```

Penjelasan:

Nilai t kecil menghasilkan t^3 yang lebih kecil → objek bergerak sangat lambat di awal, lalu mempercepat mendekati akhir.

Contoh 10: Ease-Out Cubic

```

float x0 = 100;    // posisi awal
float x1 = 500;    // posisi akhir
float y = 250;     // posisi vertikal tetap
float t = 0;       // parameter waktu
float speed = 0.01; // kecepatan interpolasi

void setup() {
  size(600, 400);
  noStroke();
  fill(255, 100, 100);
}

void draw() {
  background(30);

  // Fungsi cubic ease-out
  float f = 1 - pow(1 - t, 3);
  float x = lerp(x0, x1, f);

  ellipse(x, y, 40, 40);
  t += speed;
  if (t > 1) t = 0;
}

```

Penjelasan:

Objek bergerak cepat di awal, kemudian perlahan mendekati posisi akhir secara lembut.

Contoh 11: Ease-In-Out Cubic

```

float x0 = 100;    // posisi awal
float x1 = 500;    // posisi akhir
float y = 250;     // posisi vertikal tetap
float t = 0;       // parameter waktu
float speed = 0.01; // kecepatan interpolasi

void setup() {

```

```

size(600, 400);
noStroke();
fill(255, 100, 100);
}

void draw() {
  background(30);

  float f;
  if (t < 0.5) {
    f = 4 * pow(t, 3); // ease-in
  } else {
    f = 1 - pow(-2 * t + 2, 3) / 2; // ease-out
  }

  float x = lerp(x0, x1, f);
  ellipse(x, y, 40, 40);
  t += speed;
  if (t > 1) t = 0;
}

```

Penjelasan:

Gerakan halus di awal, cepat di tengah, dan melambat dengan lembut di akhir.

Selain cubic, ada fungsi easing lanjutan yang meniru efek fisika nyata, seperti pantulan (*bounce*) dan elastisitas (*elastic*).

A. Easing Bounce

Fungsi ini meniru efek pantulan bola ketika mengenai tanah. Grafik $f(t)$ naik-turun sebelum mencapai titik akhir.

Rumus pendekatan sederhana:

$$f(t) = \begin{cases} 7.5625t^2, & t < \frac{1}{2.75} \\ 7.5625(t - \frac{1.5}{2.75})^2 + 0.75, & t < \frac{2}{2.75} \\ 7.5625(t - \frac{2.25}{2.75})^2 + 0.9375, & t < \frac{2.5}{2.75} \\ 7.5625(t - \frac{2.625}{2.75})^2 + 0.984375, & \text{lainnya} \end{cases}$$

Contoh 12: Bounce Ease-Out

```

float x0 = 100; // posisi awal
float x1 = 500; // posisi akhir
float y = 250; // posisi vertikal tetap
float t = 0; // parameter waktu
float speed = 0.01; // kecepatan interpolasi

//Fungsi ini meniru efek pantulan bola
float bounceEaseOut(float t) {
  if (t < 1/2.75) return 7.5625*t*t;
  else if (t < 2/2.75) return 7.5625*(t - 1.5/2.75)*(t - 1.5/2.75) + 0.75;
  else if (t < 2.5/2.75) return 7.5625*(t - 2.25/2.75)*(t - 2.25/2.75) + 0.9375;
  else return 7.5625*(t - 2.625/2.75)*(t - 2.625/2.75) + 0.984375;
}

void setup() {
  size(600, 400);
  noStroke();
  fill(255, 100, 100);
}

```

```

}

void draw() {
    background(30);

    float f = bounceEaseOut(t);
    float x = lerp(x0, x1, f);

    ellipse(x, y, 40, 40);

    t += speed;
    if (t > 1) t = 0;
}

```

Penjelasan:

Objek bergerak ke kanan dan memantul beberapa kali sebelum berhenti, seperti bola yang kehilangan energi pantulan.

B. Easing Elastic

Easing elastic meniru efek getaran pegas, yaitu gerakan yang melewati batas target dan kembali berulang kali sebelum stabil.

Rumus umum (*Ease-Out Elastic*):

$$f(t) = \sin(-13\pi/2 \cdot (t + 1)) \cdot 2^{-10t} + 1$$

Contoh 13: Elastic Ease-Out

```

float x0 = 100;    // posisi awal
float x1 = 500;    // posisi akhir
float y = 250;     // posisi vertikal tetap
float t = 0;       // parameter waktu
float speed = 0.01; // kecepatan interpolasi

//Fungsi ini meniru efek getaran pegas
float elasticEaseOut(float t) {
    return sin(-13 * PI/2 * (t + 1)) * pow(2, -10 * t) + 1;
}

void setup() {
    size(600, 400);
    noStroke();
    fill(255, 150, 255);
}

void draw() {
    background(30);

    float f = elasticEaseOut(t);
    float x = lerp(x0, x1, f);

    ellipse(x, y, 40, 40);

    t += speed;
    if (t > 1) t = 0;
}

```

Penjelasan:

Objek melampaui titik akhir, berosilasi beberapa kali, lalu berhenti seperti efek spring atau pegas.

11.2.4 Spline Interpolation

Spline Interpolation adalah metode untuk menghasilkan kurva halus yang melewati serangkaian titik kontrol (*control points*). Tujuannya adalah menciptakan transisi yang lebih lembut dan realistis dibanding interpolasi linear atau polinomial sederhana. *Spline* (misalnya Catmull-Rom spline) digunakan untuk menghubungkan beberapa titik secara halus. Setiap segmen antar titik dihitung dengan kurva polinomial, sehingga hasil gerakan tidak terlihat “patah-patah”.

Dalam dunia grafika komputer dan animasi, *spline* digunakan untuk:

- Menggerakkan kamera atau karakter dengan lintasan halus
- Mengatur perubahan parameter seperti warna, skala, dan rotasi
- Membuat kurva permukaan (*surface modeling*) pada 3D modeling

Sebelum muncul *spline*, interpolasi sering dilakukan dengan polinomial tunggal derajat tinggi:

$$f(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$

Namun, pendekatan ini memiliki dua kelemahan utama:

- Runge's Phenomenon** → Polinomial tinggi menyebabkan osilasi besar di antara titik data.
- Komputasi berat** → Semakin banyak titik, semakin sulit menentukan semua koefisien polinomial.

Spline mengatasi hal ini dengan memecah kurva menjadi beberapa **segmen pendek polinomial derajat rendah (biasanya derajat 3)** yang dihubungkan dengan halus di titik sambungan (*knots*).

Spline adalah fungsi potongan (*piecewise function*):

$$S(x) = \begin{cases} S_1(x), & x_0 \leq x < x_1 \\ S_2(x), & x_1 \leq x < x_2 \\ \vdots \\ S_n(x), & x_{n-1} \leq x \leq x_n \end{cases}$$

Setiap segmen $S_i(x)$ biasanya berupa polinomial kubik (*cubic polynomial*):

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

Koefisien a_i, b_i, c_i, d_i ditentukan agar kurva:

- Melewati semua titik data,
- Memiliki turunan pertama dan kedua yang **kontinu (halus)** di titik sambungan.

Jenis-Jenis *Spline* yang Umum Digunakan:

Jenis Spline	Ciri Utama	Kelebihan	Kekurangan
Linear Spline	Hubungan garis lurus antar titik	Sederhana, cepat	Tidak halus (discontinuous slope)
Quadratic Spline	Polinomial kuadrat per segmen	Lebih halus	Derivatif kedua tidak kontinu

Cubic Spline	Polinomial kubik per segmen	Halus (kontinuitas sampai turunan kedua)	Perlu perhitungan sistem persamaan
Catmull–Rom Spline	Bentuk cubic spline yang otomatis lewat titik kontrol	Banyak dipakai di animasi	Kurva bisa terlalu lentur
B-Spline (Basis Spline)	Tidak selalu melewati titik kontrol (berbasis basis function)	Sangat stabil dan fleksibel	Kurva tidak melalui titik kontrol langsung
Bezier Spline (Cubic Bezier)	Didefinisikan dengan 4 titik (P_0 – P_3)	Umum dalam grafis vektor & animasi	Tidak langsung lewat titik kontrol tengah

Spline interpolation banyak digunakan untuk menghasilkan **gerakan halus dan realistis**, antara lain:

a. Kamera Path

Kamera mengikuti lintasan spline antar titik kunci (keyframes), menciptakan gerakan sinematik alami.

b. Gerakan Karakter

Posisi sendi (joint) atau tulang (bone) diinterpolasi dengan spline agar animasi tidak kaku.

c. Partikel

Lintasan partikel seperti asap, air, atau efek cahaya bisa mengikuti spline acak agar lebih alami.

d. Interpolasi Warna / Parameter

Spline juga bisa digunakan untuk menghaluskan perubahan warna, intensitas cahaya, atau opacity.

Visualisasi (**Contoh Catmull–Rom Spline**)

Misalkan empat titik kontrol:

$$P_0, P_1, P_2, P_3$$

Maka posisi $C(t)$ untuk $0 \leq t \leq 1$ adalah:

$$C(t) = 0.5[(2P_1) + (-P_0 + P_2)t + (2P_0 - 5P_1 + 4P_2 - P_3)t^2 + (-P_0 + 3P_1 - 3P_2 + P_3)t^3]$$

Karakteristiknya:

- Kurva *selalu* melewati P_1 dan P_2 .
- Perubahan lintasan mengikuti bentuk keseluruhan rangkaian titik kontrol.

Implementasi dalam Pseudocode:

```
// Catmull-Rom Spline antara P1 dan P2
PVector catmullRom(PVector P0, PVector P1, PVector P2, PVector P3, float t) {
    float t2 = t * t;
    float t3 = t2 * t;
    float x = 0.5 * ((2 * P1.x) + (-P0.x + P2.x) * t +
                    (2 * P0.x - 5 * P1.x + 4 * P2.x - P3.x) * t2 +
                    (-P0.x + 3 * P1.x - 3 * P2.x + P3.x) * t3);
    float y = 0.5 * ((2 * P1.y) + (-P0.y + P2.y) * t +
                    (2 * P0.y - 5 * P1.y + 4 * P2.y - P3.y) * t2 +
                    (-P0.y + 3 * P1.y - 3 * P2.y + P3.y) * t3);
    return new PVector(x, y);
}
```

Kelebihan dan Kekurangan Spline Interpolation

Kelebihan:

- Gerakan dan bentuk halus (smooth transition)
- Kontrol fleksibel melalui titik kontrol
- Dapat diterapkan pada berbagai parameter (posisi, rotasi, warna, skala, kamera)
- Tidak menimbulkan osilasi besar seperti polinomial tinggi

Kekurangan:

- Implementasi matematis lebih kompleks
- Sulit dikontrol secara intuitif (terutama untuk B-spline)
- Kadang perlu *tuning* parameter agar hasil sesuai keinginan

Contoh 14: Animasi Bola Mengikuti Catmull–Rom Spline

```
// =====
// ANIMASI GERAK BOLA DENGAN CATMULL-ROM SPLINE
// =====

PVector[] controlPoints;
float t = 0;
int segment = 1; // indeks segmen spline aktif
float speed = 0.2; // kecepatan gerak bola

void setup() {
  size(800, 600, P3D);
  smooth(8);

  // Titik kontrol (bisa diubah sesuka hati)
  controlPoints = new PVector[6];
  controlPoints[0] = new PVector(100, 400);
  controlPoints[1] = new PVector(200, 200);
  controlPoints[2] = new PVector(400, 100);
  controlPoints[3] = new PVector(600, 200);
  controlPoints[4] = new PVector(700, 400);
  controlPoints[5] = new PVector(500, 500);
}

void draw() {
  background(25);
  lights();

  // ===== Gambar garis spline (jalur lintasan) =====
  stroke(100, 200, 255);
  strokeWeight(2);
  noFill();
  beginShape();
  for (int i = 1; i < controlPoints.length - 2; i++) {
    for (float tt = 0; tt <= 1; tt += 0.02) {
      PVector p = catmullRom(controlPoints[i-1], controlPoints[i],
        controlPoints[i+1], controlPoints[i+2], tt);
      vertex(p.x, p.y);
    }
  }
  endShape();

  // ===== Gambar titik kontrol =====
  noStroke();
  for (int i = 0; i < controlPoints.length; i++) {
```



```

    fill(255, 150, 0);
    ellipse(controlPoints[i].x, controlPoints[i].y, 12, 12);
    fill(255);
    textAlign(CENTER, CENTER);
    text("P" + i, controlPoints[i].x, controlPoints[i].y - 15);
}

// ===== Hitung posisi bola pada spline =====
PVector p0 = controlPoints[segment - 1];
PVector p1 = controlPoints[segment];
PVector p2 = controlPoints[segment + 1];
PVector p3 = controlPoints[segment + 2];

PVector pos = catmullRom(p0, p1, p2, p3, t);

// ===== Gambar bola bergerak =====
pushMatrix();
translate(pos.x, pos.y, 0);
noStroke();
fill(255, 80, 100);
sphere(20);
popMatrix();

// ===== Update waktu interpolasi =====
t += speed * (1.0 / frameRate);
if (t > 1.0) {
    t = 0;
    segment++;
    if (segment >= controlPoints.length - 2) {
        segment = 1; // kembali ke awal lintasan
    }
}

// ===== Informasi tampilan =====
fill(255);
textSize(14);
textAlign(LEFT);
text("Catmull-Rom Spline Interpolation", 20, 30);
text("Segment: " + segment + " / " + (controlPoints.length - 3), 20, 50);
}

// =====
// Fungsi Catmull-Rom Spline
// =====
PVector catmullRom(PVector P0, PVector P1, PVector P2, PVector P3, float
t) {
    float t2 = t * t;
    float t3 = t2 * t;

    float x = 0.5 * ((2 * P1.x) + (-P0.x + P2.x) * t +
        (2*P0.x - 5*P1.x + 4*P2.x - P3.x) * t2 +
        (-P0.x + 3*P1.x - 3*P2.x + P3.x) * t3);
    float y = 0.5 * ((2 * P1.y) + (-P0.y + P2.y) * t +
        (2*P0.y - 5*P1.y + 4*P2.y - P3.y) * t2 +
        (-P0.y + 3*P1.y - 3*P2.y + P3.y) * t3);
    return new PVector(x, y);
}

```

Penjelasan Kode:

- a. Titik kontrol (`controlPoints`) menentukan bentuk lintasan *spline*. Kurva akan selalu melewati semua titik kecuali dua ujung pertama dan terakhir (karakteristik *Catmull-Rom*).
- b. Fungsi `catmullRom()` menghitung posisi bola di sepanjang *spline* berdasarkan parameter t (0–1).
- c. Nilai segment menentukan di antara empat titik mana bola sedang bergerak.
- d. Setelah t mencapai 1.0, animasi berpindah ke segmen berikutnya. Jika sudah mencapai akhir, bola kembali ke awal.
- e. Hasilnya adalah lintasan halus dan kontinu, berbeda jauh dari interpolasi linear biasa.

11.3 Soal Latihan

Buatlah animasi gerak objek 2D atau 3D dengan menampilkan perbandingan Teknik interpolasi linear dan interpolasi non-linear (Quadratic Interpolation dan Cubic Interpolation)!

Daftar Pustaka:

1. Foley, J.D., van Dam, A., Feiner, S.K., & Hughes, J.F. (1996). Computer Graphics: Principles and Practice. Addison-Wesley.
2. Hearn, D., & Baker, M.P. (2011). Computer Graphics with OpenGL. Pearson.
3. Reas, C., & Fry, B. (2014). Processing: A Programming Handbook for Visual Designers and Artists. MIT Press.
4. Blinn, J.F. (1998). Jim Blinn's Corner: Dirty Pixels. Morgan Kaufmann.
5. https://processing.org/reference/lerp_.html